

Chapter 8_A Practical Guide for Building Successful AI Agent

Building an effective AI agent is not about having the biggest budget or the most advanced technology. It is about understanding a few key principles that separate from failures. Success depends more on choosing the right opportunities than on technical sophistication.

When not to use an AI Agent

Tasks that require genuine human creativity or emotional intelligence should generally remain human-driven, which requires understanding the key context or making a judgment. As a level 3 agent can engage in natural language interactions, they cannot truly capture the emotional resonance needed for a compelling marketing campaign. Also, an AI agent may lack the authority to make ethical decisions, especially from a financial standpoint; it is essential to consider whether an AI agent has the appropriate level of authority for the task it is assigned.

The 3 Circles of Agentic Opportunities

- **Circle 1: High Impact - Will it matter?**
 - If we automate this process, will it make a meaningful difference? Impact is not about saving time; it is almost about what that saved time enables our organization to do.
 - The highest-impact opportunities often are not the most complex processes.
- **Circle 2: Feasibility - Can it be Done?**
 - About whether current AI agent technology can actually handle the task effectively and safely.
 - Characteristic:
 - Clear, consistent rules for making decisions
 - accessible data and system
 - definable success criteria
 - manageable consequences if something goes wrong
 - ability to verify results before they impact operation
 - The key is to look for processes where you can explain the rules without using phrases like "it depends". or "but sometimes" too often. The more exceptions and judgements call a process requires, the less suitable it is for your first AI agent project.
- **Circle 3: Effort - is it worth it?**
 - Characteristics:
 - The process is well-documented
 - the team is ready and willing to adapt

- you can start small and scale up
- the potential benefits clearly justify the investment
- you can implement without disrupting core operations

Finding the Agent Opportunities

- Step 1: Task Inventory: the goal is to create a comprehensive list of recurring tasks that consume significant time or create bottlenecks in the operation
 - Ask questions:
 - What tasks do you find yourself doing repeatedly throughout the week?
 - Which activities prevent you from focusing on more strategic work
 - What processes consistently create bottlenecks in our operations
 - Which routine tasks require the most oversight to prevent errors
- Step 2: Impact assessment

Criteria	Score on a scale from 1 to 5		
Time Investment	1: Less than an hour per week	3: Several hours per week	5: Multiple hours daily
Strategic Value of Freed Time	1: Limited alternative use of time	3: Moderate strategic value	5: High-value strategic activities blocked
Error Reduction Potential	1: Few errors occur	3: Occasional significant errors	5: Frequent or costly errors
Scalability Impact	1: One-off task	3: Moderately repeatable	5: Highly scalable across the organization

Table 8.1: Scoring the four criteria of impact assessment (Source: © Bornet et al.)

- Step 3: Feasibility Assessment

Criteria	Score 1	Score 3	Score 5
Process Standardization	1: Process is largely ad-hoc with no standard approach.	3: Basic documentation exists but relies heavily on employee experience.	5: The process is fully documented with clear steps, decision rules, and exceptions.
Data and System Access	1: Essential data is locked in legacy systems or paper-based.	3: Data is available but requires significant preparation.	5: All data is structured, and systems have modern APIs.

Table 8.2: Scoring the two criteria of feasibility assessment (Source: © Bornet et al.)

- Step 4: Implementation effort
 - Score 5: The task can be automated with standard tools that require little or no customization
 - Score 4: Some customizations are needed, but it relies on widely used technologies
 - Score 3: The solution requires significant custom development, such as scripting
 - Score 2: Complex integrations or new technology adoption are required
 - Score 1: The task demands cutting-edge solutions or extensive research and development (R&D)

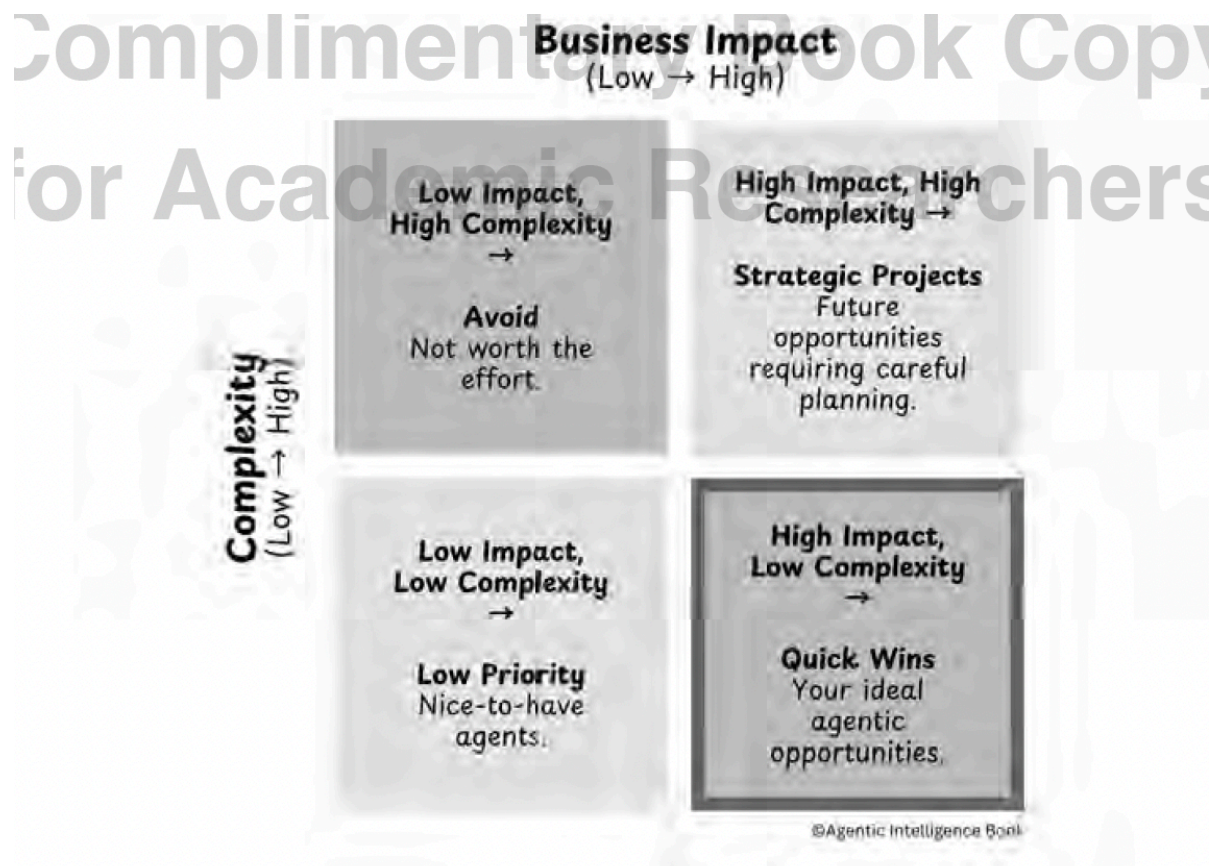


Figure 8.2: The Agentic AI Prioritization Matrix (Source: © Bornet et al.)

The deterministic model and the probabilistic model

These two can work hand in hand (i.e., the deterministic can be used for clear accuracy, like summarization,... and the probabilistic model can be used for data interpretation)

Design Patterns

1. Principle: Start with an end in mind

Organizations often jump straight to solution design without defining what success looks like

Defining clear success is essential for three key reasons:

- It provides clear targets for the development team and helps prioritize features
- It establishes benchmarks for measuring the return on investment and system performance
- It helps manage stakeholders' expectations and build confidence in the new system

Success criteria should include

- Quantitative metrics might include efficiency gains, error reduction rates, or specific performance targets
- Qualitative metrics often focus on user experience, output quality, and system reliability.

2. Understand the current workflow

- It reveals hidden complexities and dependencies that might not be apparent at first glance
- It helps identify potential bottlenecks and inefficiencies that the new system should address.
- It ensures that critical business rules and expectations are not overlooked in the automation process.

3. Design the target process

Don't just automate; rethink and optimize for the best possible outcome.

4. Choose the right architecture

Should you build one complex agent to handle everything or create a team of specialized agents?

Specialized components often work better than complex, monolithic ones.
"One agent, one tool"

5. Build in Human-AI collaboration

1. Strategic Decision-making: human excel at understanding context and making nuanced judgements
2. Quality Assurance: Providing oversight and catching subtle errors
3. Exception handling: Managing unexpected situations that fall outside normal parameters
4. Continuous Improvement: Identifying opportunities for system enhancement

AI Platforms

- **No-code platforms:**
 - bizway
 - N8N
 - Relavance AI
 - Enables business users to create a functional AI agent through an intuitive interface without writing any code
- **Full-code platforms:**
 - Provide the highest control and flexibility, but require significant development success
 - can fine-tune every aspect of their AI agent's behavior, from decision-making processes to data-handling protocols.
 - LangGraph
- **Low-code platforms:**
 - WatsonXAssistant, Agentforce, UniPath, Microsoft Copilot's Agent builder
 - the balanced approach

Avoid overthinking: No platform is perfect, and waiting for the "ideal" choice can lead to lost opportunities.

The A.G.E.N.T Framework

Components:

- **Agent Identity (who is the agent?)**
- **Gear & brain (What power does the agent have?)**
 - Temperature: controls how deterministic or creative the AI's responses will be, impacting both the agent's reliability and flexibility.
 - 0 - 0.3 = accuracy is crucial, such as summarization agents, research assistants, and legal AI tools
 - 0.7 - 1 = make the AI more creative, is useful for brainstorming, marketing copy, writing creative content, where originality is more important than strict accuracy.
 - **Circuit Breakers:** stop executing if repeated failures occur, rather than continuing to send faulty requests. For example, if an api fail 3 times in a row, the agent should pause and notify an administrator instead of retrying indefinitely. We can set a quality threshold for the agent to break the circuit
 - **Fallback system:** Ensure continuity by switching to a backup process when something goes wrong. For example, if the new API is down, the AI can pull a headline from a secondary source instead of returning an error.
- **Execution & workflow (How does the agent work?)**
 - **Defining output and input**
 - Strict input validation: Ensure the agent can process only the format it is designed for
 - Clear output definitions: the receiving system should always know what to expect
 - With the circuit breakers and structured error handling, the agents can detect issues early, prevent cascading failures, and ensure graceful recovery.
 - Error Handling:
 - Automated recovery: the system automatically retries failed processes, waiting longer between each attempt to avoid overload while increasing the chances of success.
 - Graceful Degradation: if a function fails repeatedly, the AI reduces complexity instead of stopping entirely. For instance, if a summarization agent encounters errors, it can still forward article links and metadata rather than fail completely
 - Human Escalation: if the AI cannot resolve that problem with confidence, it flags a human, providing relevant context and suggested resolutions rather than simply failing
 - **Activation criteria:** User input, API calls, Schedule Execution

- **Navigation & Rules (How does the agent make decisions?)**
 - **Processing Systems:**
 1. **Filtering mechanism:** the agent must distinguish between relevant and irrelevant data. A research AI, for instance, should prioritize a peer-reviewed paper over an unverified blog post
 2. **Prioritization logic:** Some tasks are more urgent or valuable than others. A customer support AI should escalate urgent complaints before handling general inquiries.
 3. **Processing limit:** Overloading an agent with too much data at once slows it down and increases costs. If an AI retrieves thousands of articles, it should pre-filter based on key parameters rather than analyzing everything.
- **Testing & trusting (How do we improve and scale the agent?)**
 - LangChain's testing suite
 - AI model benchmarking from HuggingFace
 - Adversarial testing frameworks from OpenAI
 - Collective feedbacks:
 - User feedback mechanism
 - Log Analysis: record input, outputs, and decision paths to detect recurring error or inefficiency
 - Behavior tracking: identify whether users abandon interactions, request clarifications, or get stuck frequently.
 - Observability tools: LangSmith, OpenTelemetry for tracking model behavior and user interaction